Documentación de diseño

Diagramas de clases

Diagrama de alto nivel

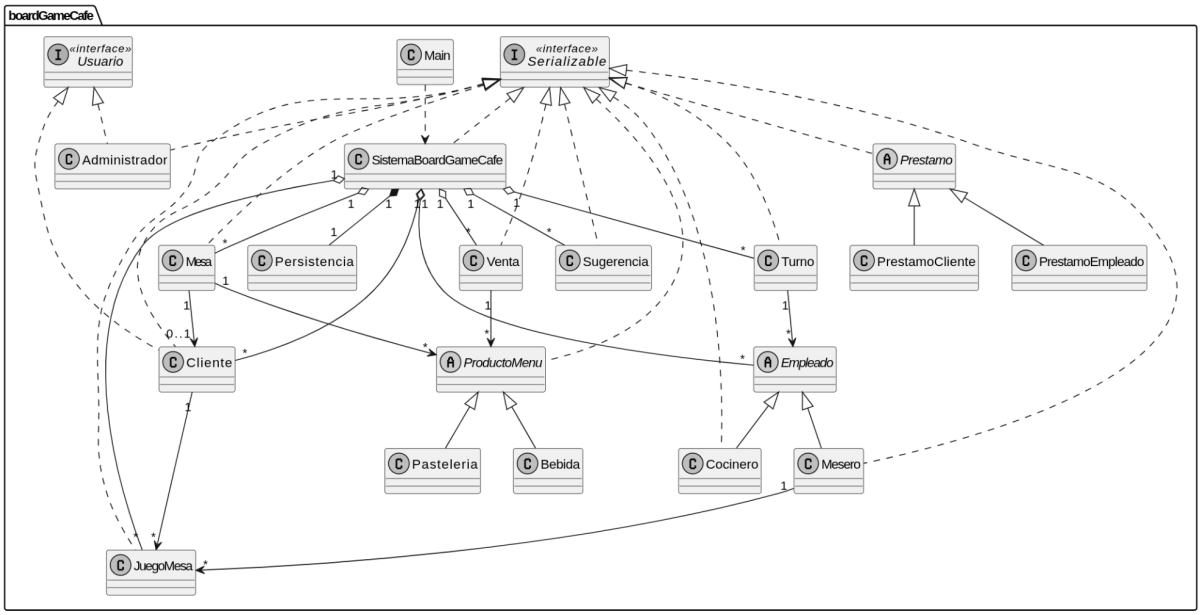
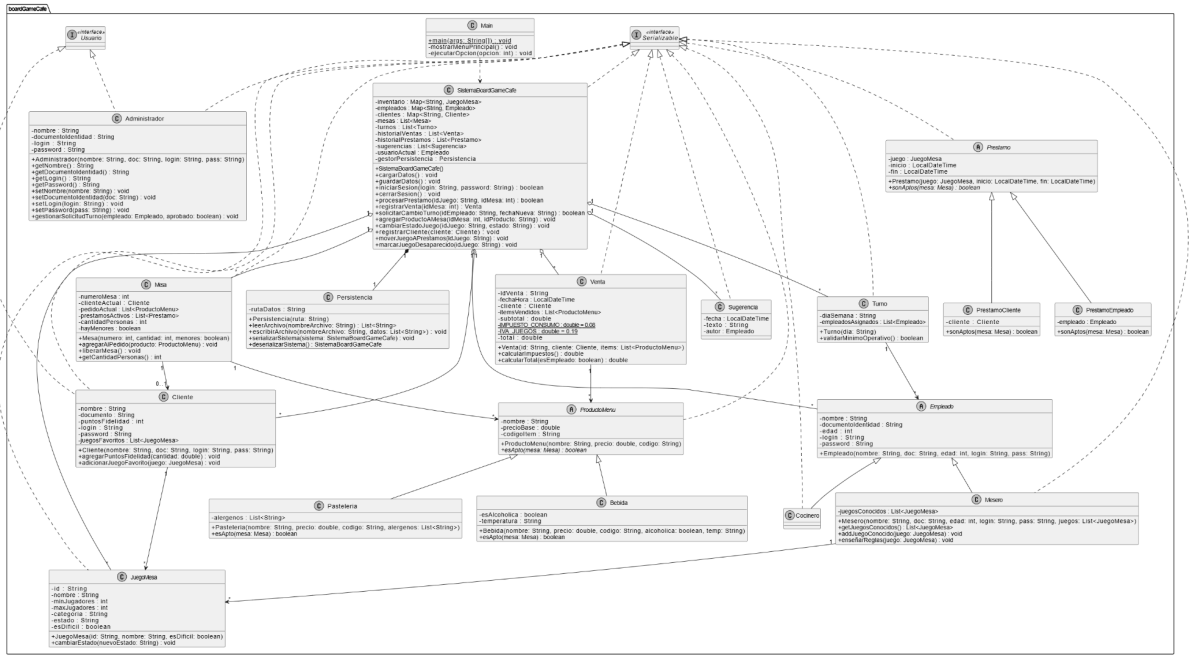


Diagrama detallado



Justificaciones y proceso de diseño

La evolución del modelo conceptual inicial hacia el diseño de software definitivo requirió una serie de reestructuraciones arquitectónicas fundamentadas en los principios de diseño orientado a objetos. El objetivo principal de estas modificaciones fue garantizar un sistema con alta cohesión, bajo acoplamiento y responsabilidades claramente delimitadas. A continuación, se detallan y justifican las decisiones clave tomadas durante este proceso:

1. Implementación de un Controlador Central (Patrón Fachada)

Decisión: Se introdujo la clase SistemaBoardGameCafe como coordinador central de la aplicación, eliminando la ejecución de procesos directamente desde las clases que representan a los actores (Administrador, Mesero, etc.). Justificación: En el modelo inicial, se corría el riesgo de acoplar la interfaz de usuario con las entidades del dominio. Al introducir un controlador central, se establece una capa intermedia (Fachada) que recibe las peticiones de la vista (Main), verifica el estado de la sesión (usuarioActual) y orquesta las validaciones de negocio. Esto garantiza que las entidades de datos (usuarios) no asuman responsabilidades de ejecución de sistema, manteniendo una arquitectura limpia y facilitando futuras modificaciones en la interfaz sin alterar la lógica central.

2. Consolidación de Estructuras de Datos y Eliminación de Clases Contenedoras

Decisión: Se eliminaron del diseño conceptual clases como Inventario, Historial o GestorMesas. Sus datos correspondientes se transformaron en colecciones (estructuras List y Map) administradas directamente por el controlador. Justificación: Las clases eliminadas representaban un problema de diseño conocido como "clases anémicas", es decir, objetos que únicamente almacenan listas de datos pero carecen de comportamiento o lógica de negocio sustancial. Al trasladar estas colecciones como atributos del SistemaBoardGameCafe, se simplifica la jerarquía del sistema, se centraliza el acceso a la información y se evitan saltos de memoria innecesarios al momento de buscar o iterar sobre los elementos.

3. Corrección del Ciclo de Vida y Relaciones Estáticas (Mesa y Cliente)

Decisión: Se modificó la direccionalidad de la relación entre clientes y mesas. Se determinó que la entidad Mesa mantenga una referencia temporal al Cliente (clienteActual), y no a la inversa. Justificación: Esta decisión se basa en el ciclo de vida de los objetos dentro del restaurante. La ocupación de un espacio es un estado efímero y propio de la Mesa. Por el contrario, un Cliente es una entidad persistente cuyo propósito es almacenar historial, credenciales y puntos de fidelidad. Asignar una mesa como atributo de un cliente implicaría contaminar una entidad permanente con datos de sesión temporales, rompiendo el principio de encapsulamiento estricto.

4. Cohesión y Delegación de Cálculos Internos

Decisión: Se decidió que la clase controladora orqueste los procesos, pero que los cálculos matemáticos y lógicos se ejecuten dentro de las entidades correspondientes (por ejemplo, calcularImpuestos() dentro de Venta o validarMinimoOperativo() dentro de Turno). Justificación: Esta distribución respeta el principio fundamental de encapsulamiento. Una entidad debe ser responsable de manipular los datos que contiene de manera privada. Si el controlador extrajera los valores numéricos de una Venta para calcular el IVA externamente, se generaría un alto acoplamiento. Al delegar estas operaciones a las clases propietarias de los atributos, se asegura que cualquier cambio futuro en las normativas de impuestos o reglas logísticas solo requiera modificaciones locales en la entidad afectada, protegiendo al resto del sistema.

Persistencia

La persistencia en el sistema la implementamos usando serialización en Java, ya que permite guardar todo el estado del programa en un solo archivo (sistema.dat) y recuperarlo después sin perder la información. Elegimos este método porque es mas sencillo de implementar, no requiere bases de datos externas y permite almacenar directamente objetos completos con todas sus relaciones (como clientes, juegos y mesas), lo cual facilita mucho el desarrollo del proyecto. Además, al centralizar la lógica en una clase de persistencia con métodos para guardar y cargar, se mantiene el código organizado y fácil de mantener. También se manejan

errores para que, si el archivo no existe, el sistema cree uno nuevo sin fallar. Aunque este enfoque no es el más eficiente para sistemas grandes, lo consideramos adecuado para el alcance del proyecto porque teóricamente cumple correctamente con la necesidad de almacenar y recuperar datos de forma simple y funcional.

¿Como funciona?
En este caso la persistencia implementada funciona guardando el objeto principal del sistema (SistemaBoardGameCafe) directamente en un archivo mediante serialización y cuando se llama al método guardarSistema, se utiliza un ObjectOutputStream que toma todo el objeto y lo convierte en una secuencia de bytes, incluyendo todos los objetos que contiene internamente (clientes, juegos, mesas, préstamos, etc.), y los escribe en el archivo sistema.dat. Luego, cuando se ejecuta cargarSistema, un ObjectInputStream lee ese archivo y reconstruye el objeto completo en memoria, manteniendo sus datos y relaciones tal como estaban al momento de guardarse. Si el archivo no existe o ocurre algún error al leerlo, el sistema crea una nueva instancia vacía, lo que permite que el programa siga funcionando sin fallar. Este proceso permite simular que los datos “se guardan” entre ejecuciones del programa, ya que al cerrarlo y volverlo a abrir, la información previamente almacenada se recupera automáticamente.

Responsabilidades

Una vez realizado el diseño y sus respectivos diagramas de clases, decidimos realizar los diagramas de secuencia de las 3 funcionalidades críticas del sistema. Para esto, primero asignamos sus responsabilidades a cada componente del sistema.

Asignación de responsabilidades

1. Interfaces del Sistema

- Interfaz Usuario: Su responsabilidad es definir un contrato común para todas las entidades físicas que interactúan con el software (Empleados, Administradores y Clientes). Esto permite al controlador principal manejar el estado de la sesión (usuarioActual) de forma polimórfica sin acoplarse a una clase concreta.
- Interfaz Serializable: Su responsabilidad es habilitar la conversión de los objetos del sistema en flujos de bytes (mecanismo nativo de Java) para facilitar su guardado y recuperación en disco.

2. VIsta

- Clase Main: Es la única clase responsable de la interacción directa con el actor humano. Se encarga de imprimir los menús en la consola, capturar las entradas por teclado y realizar los llamados correspondientes a los métodos del controlador central. No contiene lógica de negocio.

3. Capa de Control y Lógica de Aplicación

- Clase SistemaBoardGameCafe: Actúa el controlador central del sistema. Sus responsabilidades incluyen:
    - Centralizar y mantener en memoria las colecciones de datos (inventarios, historiales, usuarios, mesas).
    - Orquestar la ejecución de los casos de uso críticos (préstamos, ventas, gestión de turnos) delegando cálculos específicos a las entidades correspondientes.
    - Gestionar el ciclo de vida de la sesión (iniciar/cerrar sesión) basándose en la interfaz Usuario.

4. Capa de Persistencia de Datos

- Clase Persistencia: Aplicando el principio de responsabilidad única, esta clase se encarga exclusivamente de las operaciones de guardar y cargar información. Su responsabilidad es leer, escribir, serializar y deserializar el estado del SistemaBoardGameCafe hacia y desde los archivos físicos, aislando esta complejidad del resto del dominio.

5. Entidades de Dominio: Productos y Menú

- Clase abstracta ProductoMenu: Responsable de definir los atributos comunes y el contrato de validación de todos los ítems consumibles de la cafetería.
- Clases Bebida y Pasteleria: Responsables de almacenar sus características específicas (contenido alcohólico, temperatura, alérgenos). Se determina mediante la implementación de su propio método esApto(Mesa), si pueden ser despachadas a una mesa bajo las restricciones de seguridad vigentes.

6. Entidades de Dominio: Operación y Transacciones

- Clase Mesa: Responsable de gestionar el estado temporal del servicio a un grupo de personas. Encapsula quién es el cliente sentado, cuántas personas hay, si hay menores y mantiene la lista de los productos consumidos (pedidoActual) antes de la facturación.
- Clase Venta: Responsable de encapsular la transacción comercial final. Actúa como la responsable de calcular independientemente el impuesto al consumo (8% para alimentos), el IVA (19% para juegos) y el total a pagar, aplicando descuentos cuando corresponda.
- Clase JuegoMesa: Responsable de mantener la información técnica del catálogo (rango de jugadores, dificultad) y de gestionar sus cambios de estado lógicos (disponible, prestado, desaparecido).

7. Entidades de Dominio: Personal y Clientes

- Clases Empleado (Abstracta), Cocinero y Administrador: Responsables de mantener los datos contractuales y credenciales de acceso del personal. El Administrador incluye la

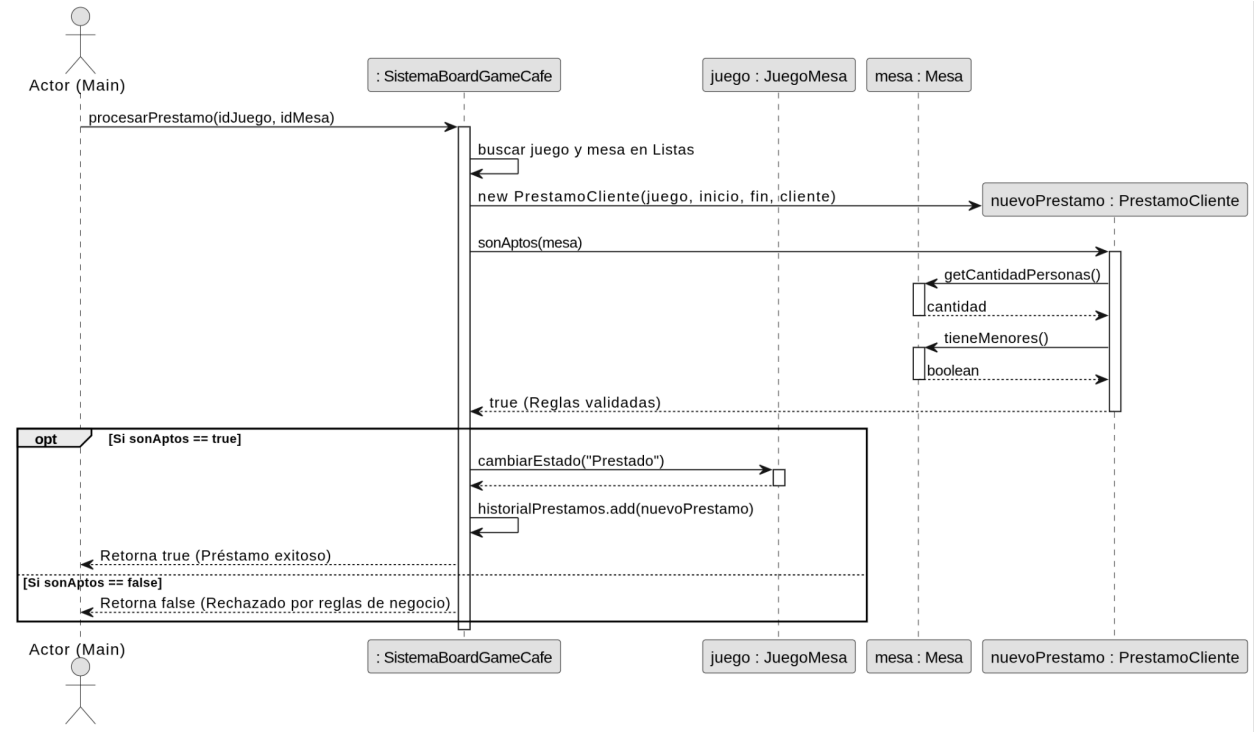
responsabilidad operativa de procesar aprobaciones o rechazos sobre la gestión del sistema.

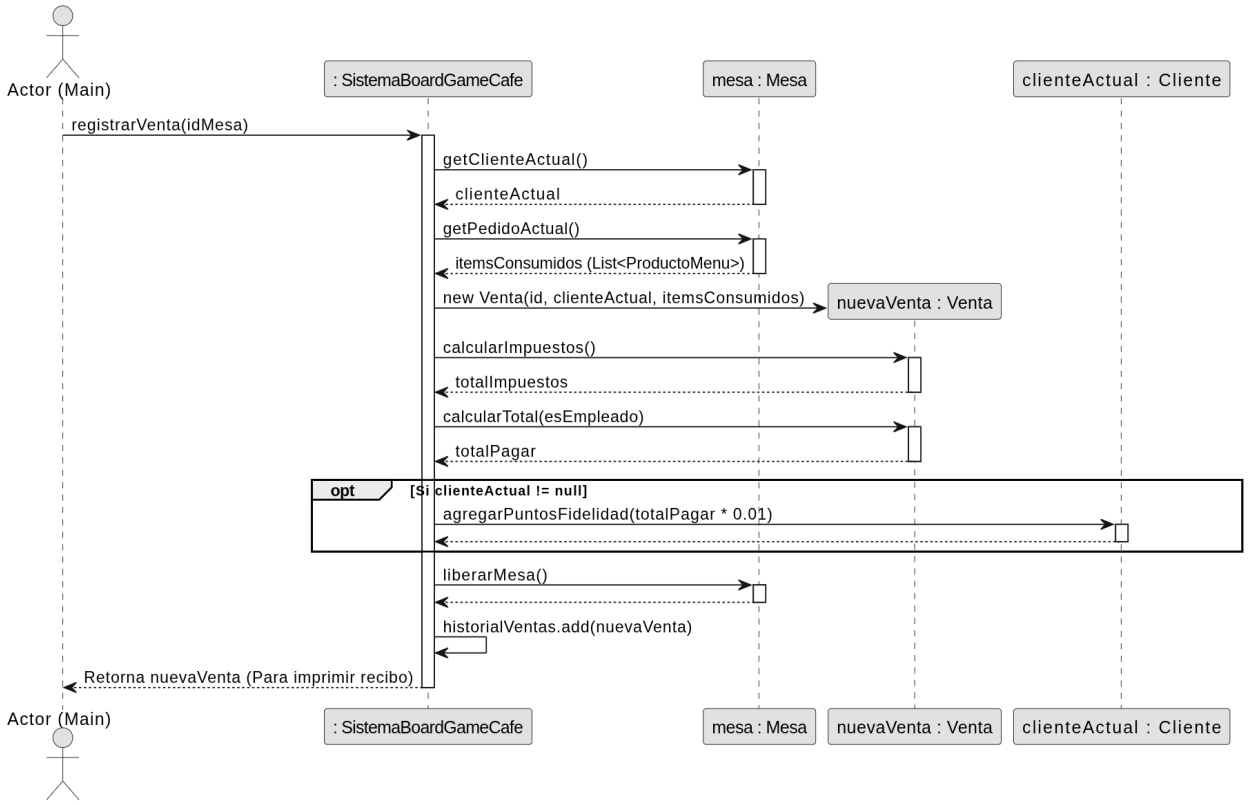
- Clase Mesero: Además de ser un empleado, tiene la responsabilidad específica de conocer y gestionar la lista de juegos cuyas reglas está capacitado para enseñar a los clientes.
- Clase Cliente: Responsable de almacenar la información personal y las credenciales de los visitantes frecuentes. Gestiona internamente la acumulación de sus puntosFidelidad y el registro de sus juegosFavoritos.

8. Entidades de Dominio: Logística

- Clase Turno: Responsable de agrupar los trabajadores asignados a un día específico. Es la encargada de validar (validarMinimoOperativo()) que la suma de sus empleados cumpla con las políticas de funcionamiento del café (mínimo 1 cocinero y 2 meseros).
- Clase abstracta Prestamo y derivadas (PrestamoCliente, PrestamoEmpleado): Responsables de trazar la línea de tiempo (inicio y fin) de un juego en uso. Las clases derivadas aplican el polimorfismo para evaluar (sonAptos(Mesa)) si las condiciones de la mesa destino cumplen los requisitos de edad y capacidad según el tipo de usuario que solicita el juego.
- Clase Sugerencia: Actúa como un contenedor de datos (Data Object) responsable de unificar el texto de una recomendación con su fecha de creación y el autor que la redactó.

Diagramas de secuencia





Consideraciones finales

1. Los diagramas se pueden apreciar mejor revisándolos en la carpeta “Entrega 2/diagrams” del repositorio.
2. En esta entrega intentamos implementar la persistencia y el main de la mejor manera posible, sin embargo, en dicha implementación tuvimos dificultades las cuales pueden obstruir el correcto funcionamiento del sistema, sin embargo entendemos la lógica y funcionamiento de estas.

Declaración sobre el Uso de Herramientas de Inteligencia Artificial

Para la elaboración de la presente entrega, se utilizaron modelos de lenguaje de inteligencia artificial (específicamente Gemini y ChatGPT) de manera ética y estrictamente como herramientas de apoyo metodológico. Su uso se limitó a las siguientes tareas:

- Aclaración de conceptos teóricos.
- Estructuración formal del documento de diseño.
- Refinamiento de redacción, ortografía y estilo técnico.

Se deja expresa constancia de que la toma de decisiones arquitectónicas, el proceso esencial de diseño orientado a objetos (incluyendo la definición de clases, métodos y responsabilidades) y la construcción lógica del código fuente alojado en el repositorio, fueron realizados y

analizados íntegramente por los autores del proyecto. La inteligencia artificial no fue utilizada para generar la lógica de negocio ni para programar la solución final entregada.